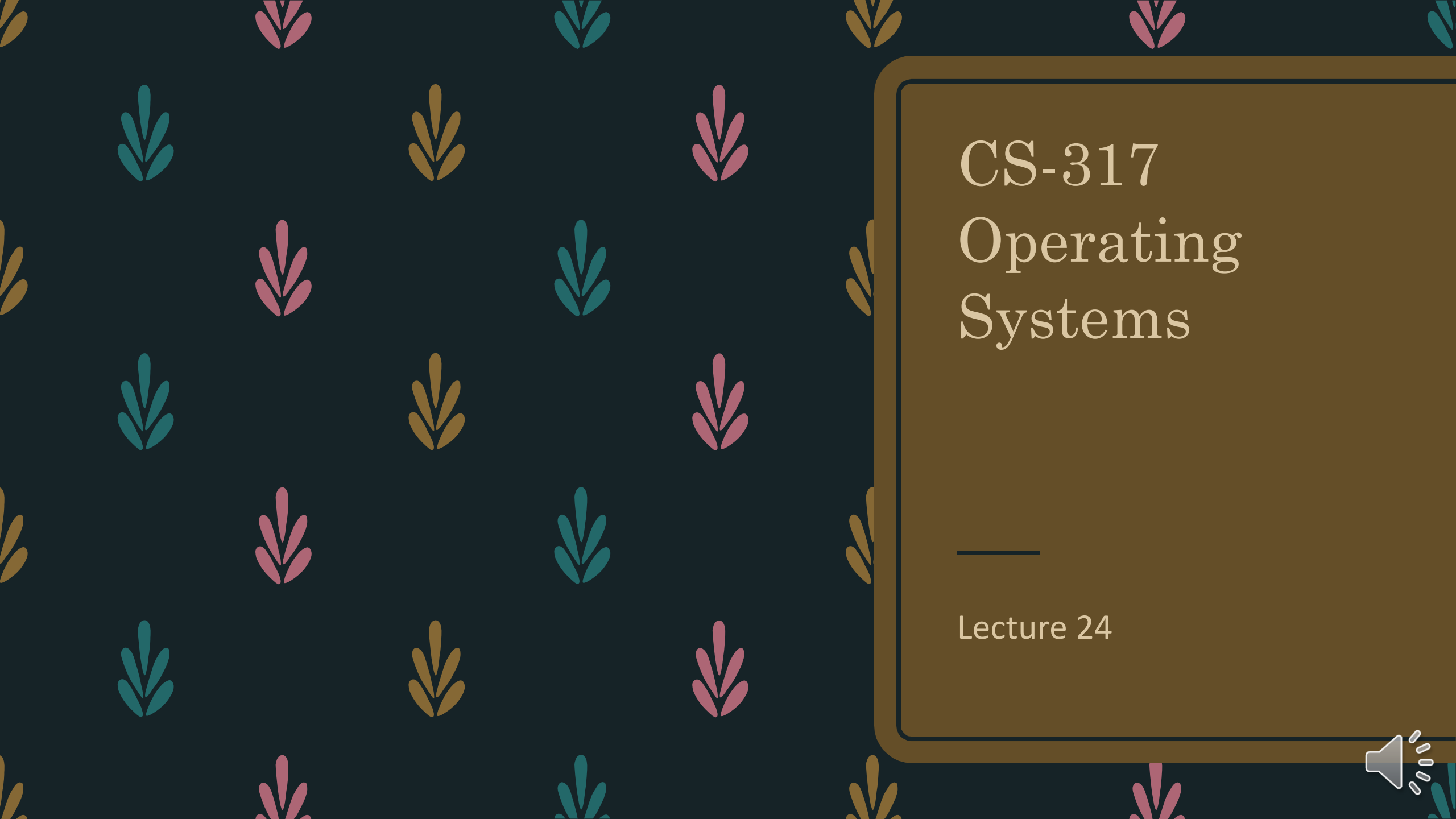




CS-317

Operating Systems

Lecture 24



Deadlocks

Book 1 Chapter 6



Principles of Deadlock

Definition and conditions



Definition of deadlock

- Deadlock can be defined as the permanent blocking of a set of processes that either compete for system resources or communicate with each other.
- A set of processes is deadlocked when each process in the set is blocked awaiting an event (typically the freeing up of some requested resource) that can only be triggered by another blocked process in the set.
- Deadlock is permanent because none of the events on which the participants of a deadlock are waiting ever happen.
- Unlike other problems in concurrent process management, there is no efficient solution for deadlock in the general case.



A traffic deadlock

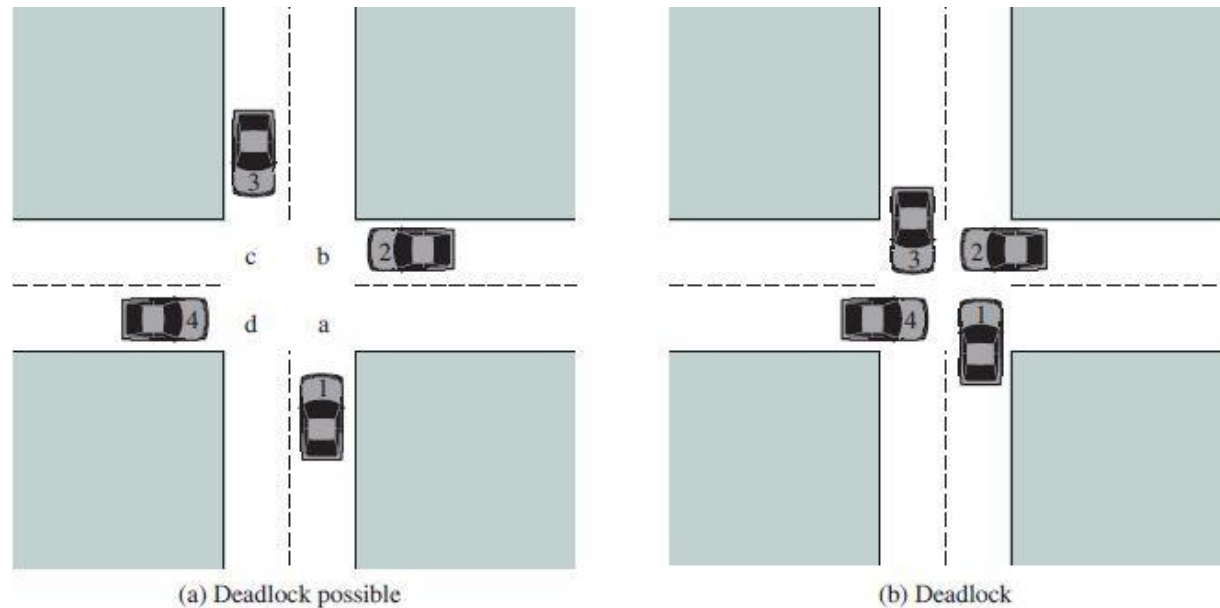


Figure 6.1 Illustration of Deadlock

- Each car wants to move straight ahead.
- In the figure on the left, deadlock can be avoided if any one of the cars stops where it is. So the figure marked (a) shows a potential deadlock.
- In the figure on the right, the cars have continued forward until there is no room further. This is a deadlock.



Deadlock of processes over resources

- Two processes, P and Q, have the general form given in the figure above.
- A joint progress diagram, illustrates the progress of two processes competing for two resources.
- Each process needs exclusive use of both resources for a certain period.
- In Figure 6.2, the x-axis represents progress in the execution of P and the y-axis represents progress in the execution of Q.

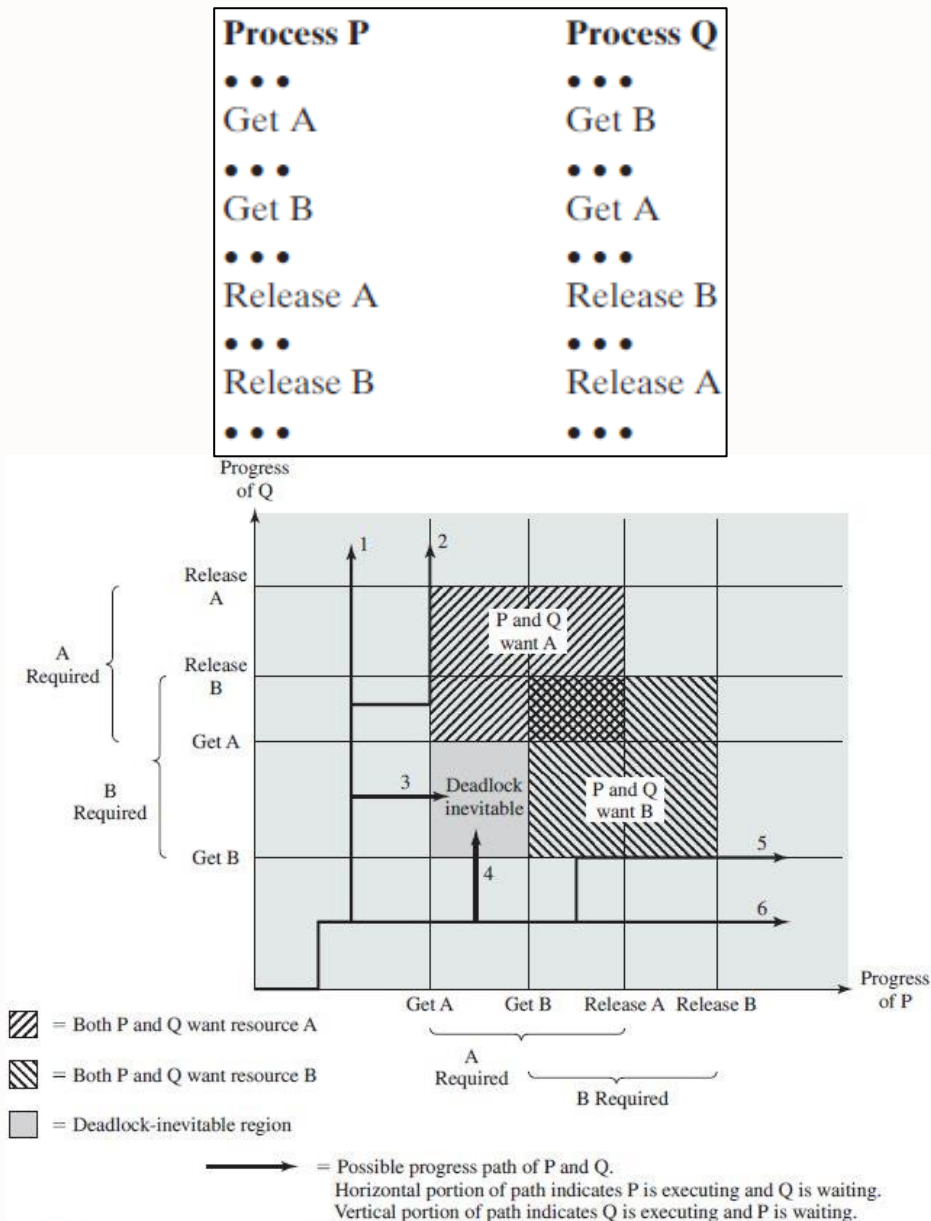


Figure 6.2 Example of Deadlock



Deadlock of processes over resources

path 1 discussed

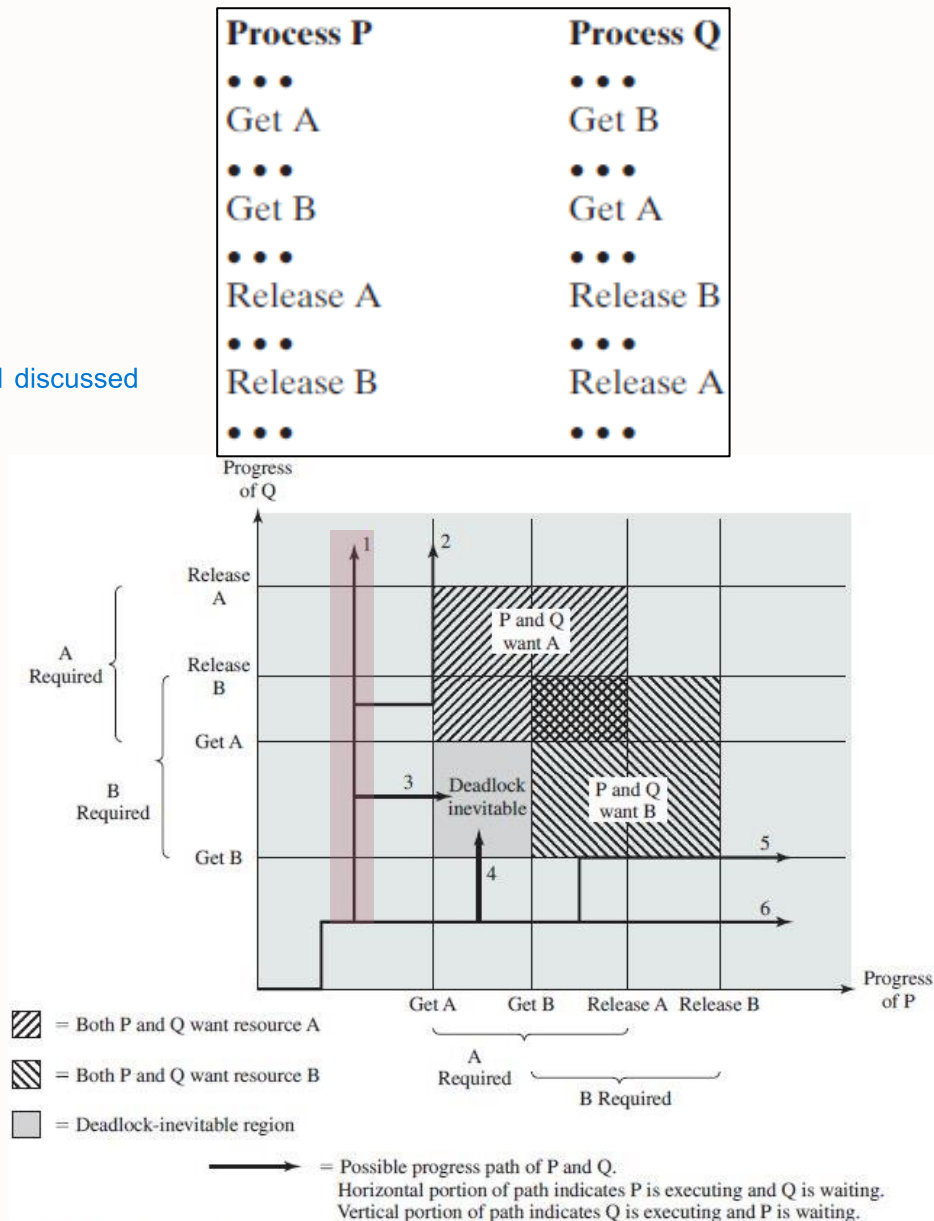


Figure 6.2 Example of Deadlock

- For a uniprocessor system, only one process at a time may execute.
- The path consists of alternating horizontal and vertical segments, with a horizontal segment representing a period when P executes, and Q waits and a vertical segment representing a period when Q executes, and P waits.
- The figure shows six different execution paths.
 - Q acquires B and then A and then releases B and A. When P resumes execution, it will be able to acquire both resources.



Deadlock of processes over resources

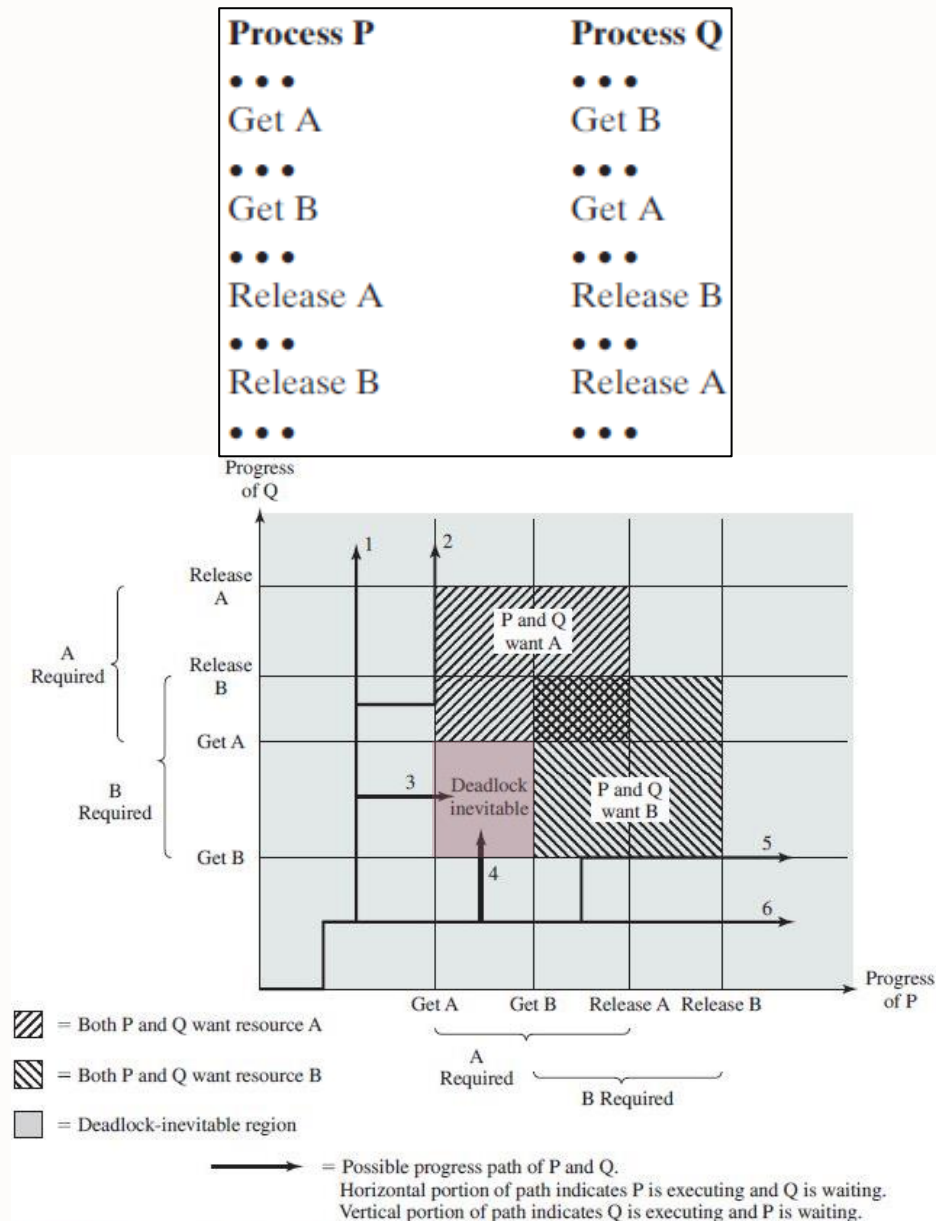


Figure 6.2 Example of Deadlock

- The pink-shaded area can be referred to as a fatal region.
- If an execution path enters this fatal region, then deadlock is inevitable.
- The existence of a fatal region depends on the logic of the two processes.
- Whether or not deadlock occurs depends on both the dynamics of the execution and on the details of the application.



Deadlock of processes over resources

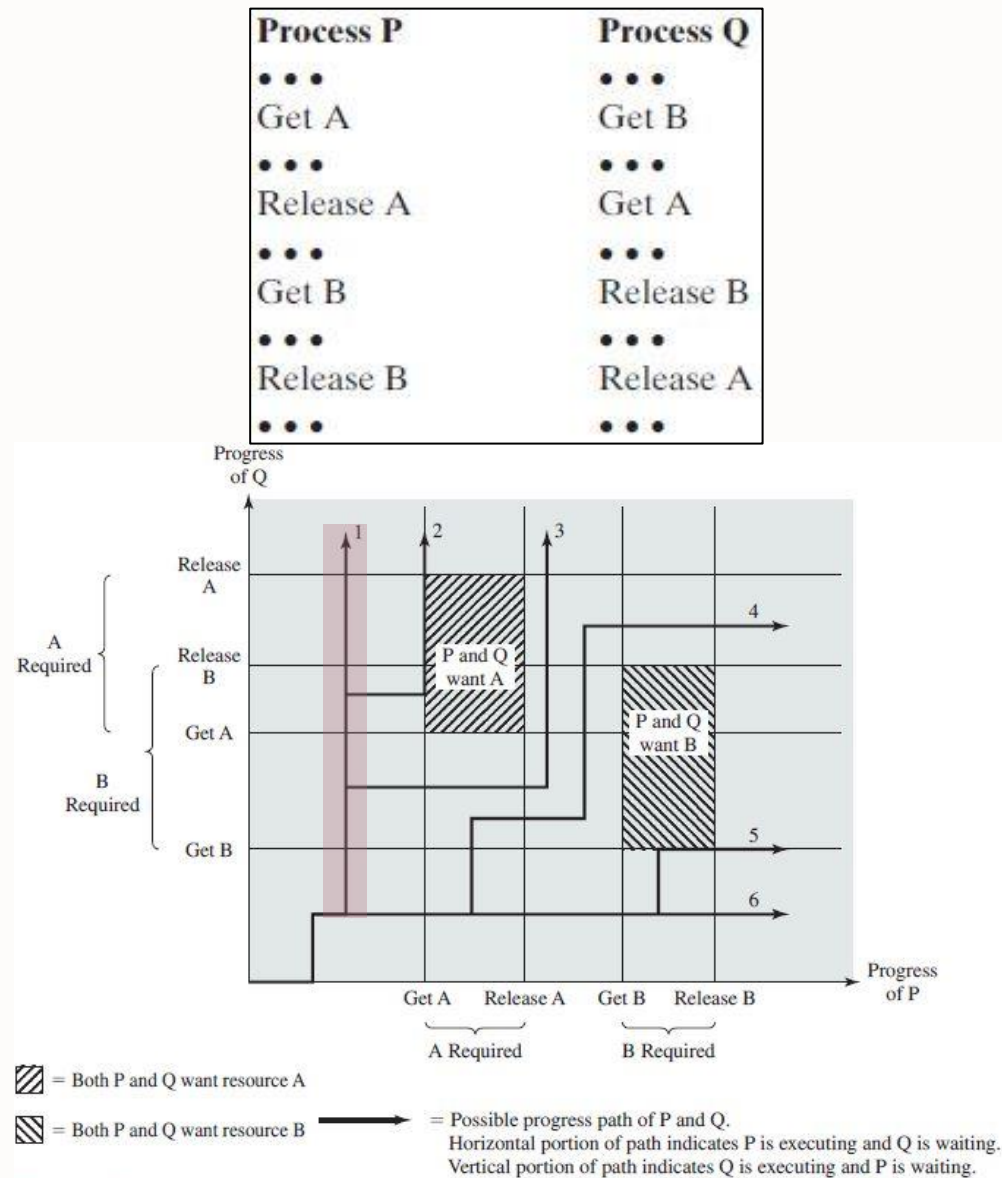


Figure 6.3 Example of No Deadlock [BACO03]

- Two processes, P and Q, have the general form given in the figure above.
- If we follow line marked 1,
 - Q gets B, then gets A, then releases B, then releases A. Now P can execute.



Deadlock of processes over resources

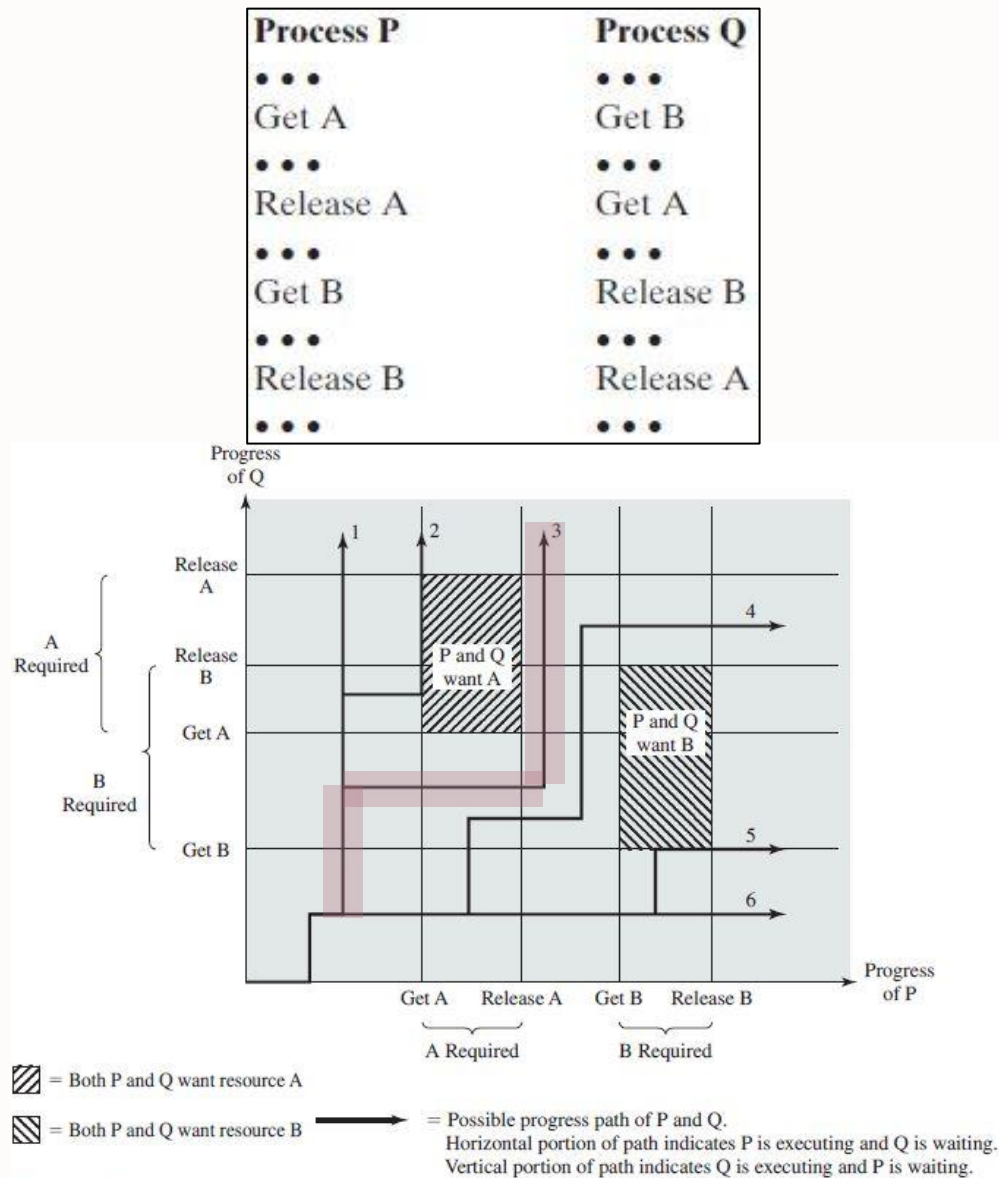


Figure 6.3 Example of No Deadlock [BACO03]

- Two processes, P and Q, have the general form given in the figure above.
- If we follow line marked 3,
 - Q gets B.
 - Then P starts executing and gets A. P releases A.
 - Then Q resumes execution and gets A, then releases B, then releases A. Now P can execute.



Deadlock of processes over resources

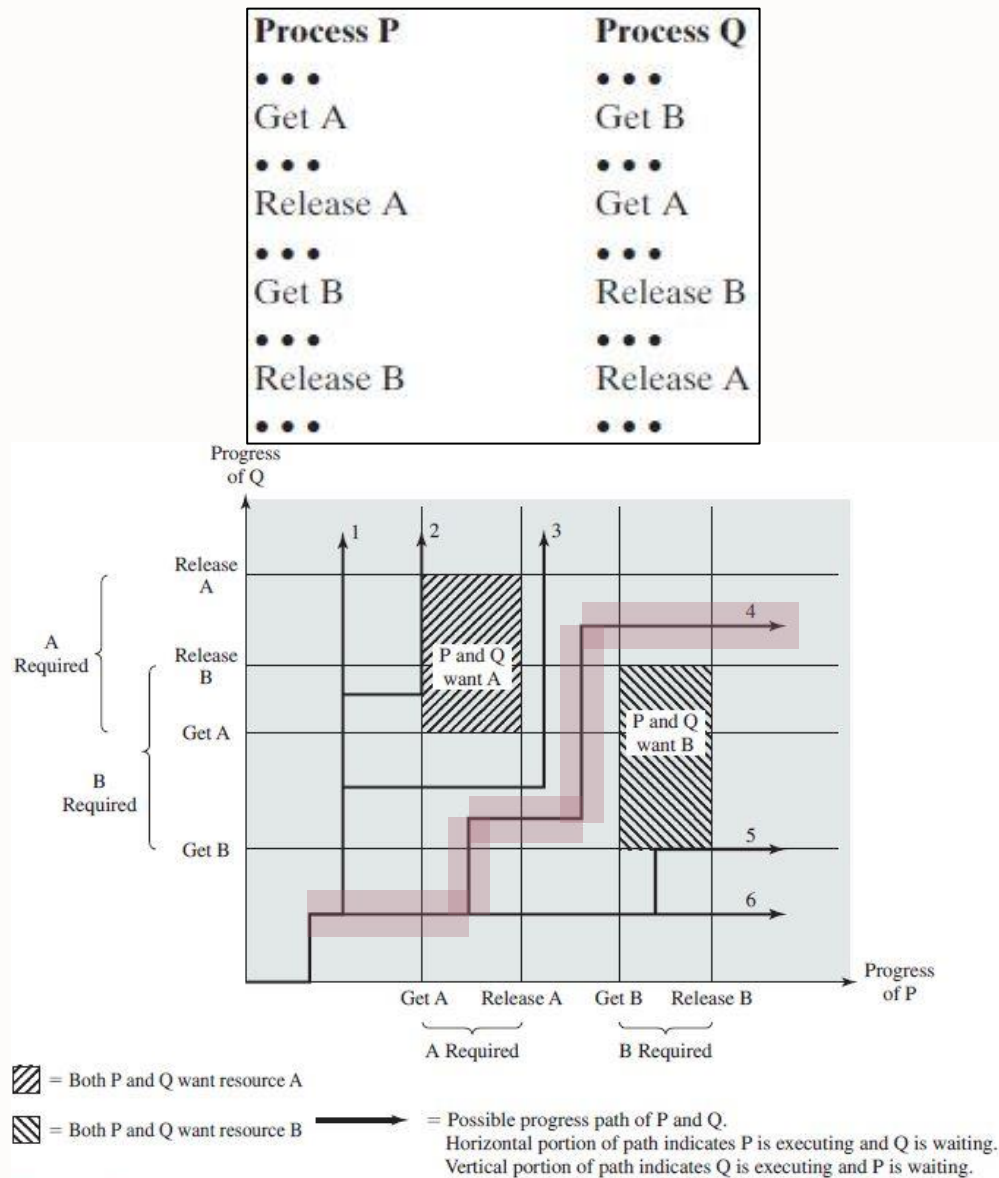
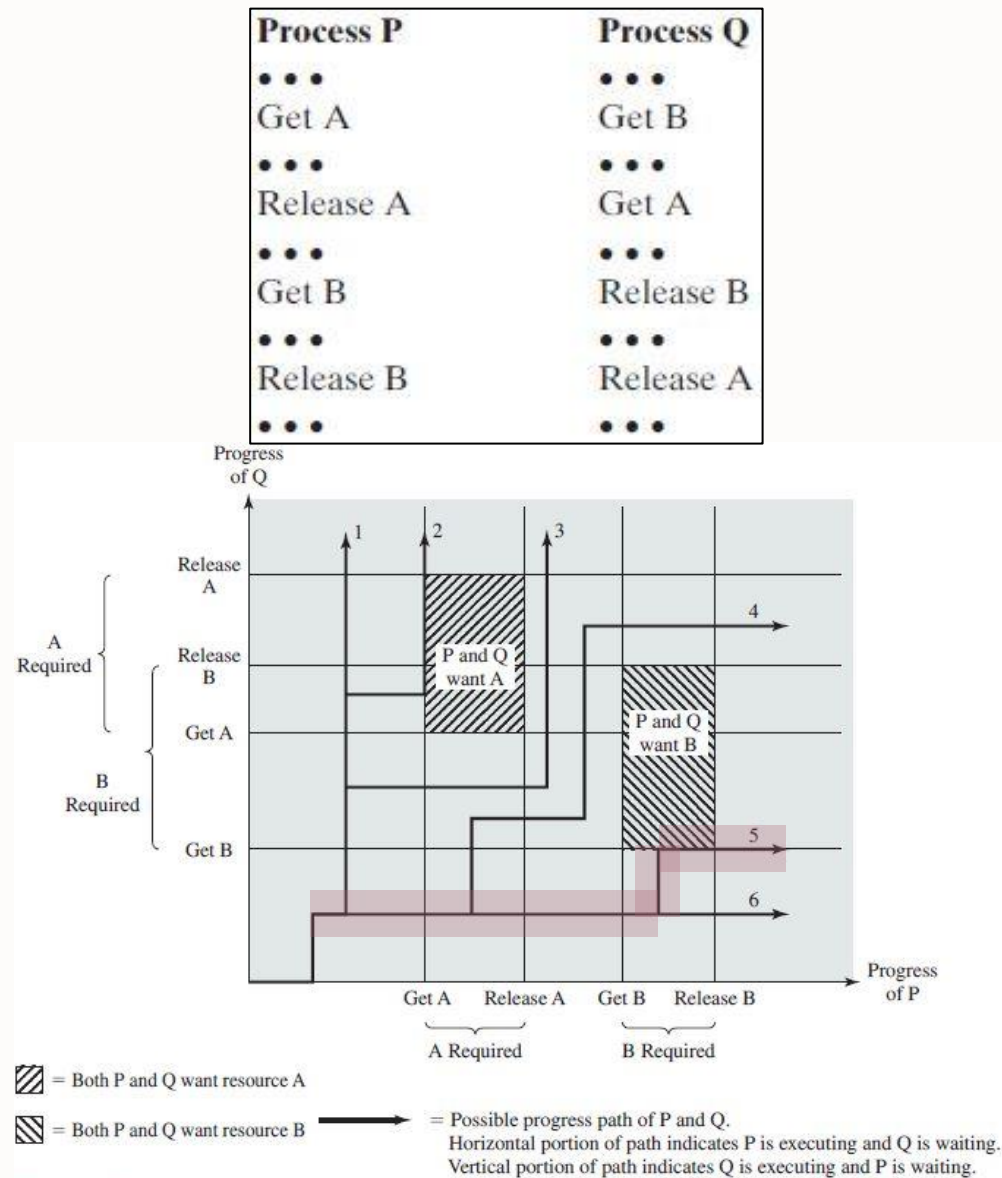


Figure 6.3 Example of No Deadlock [BACO03]

- Two processes, P and Q, have the general form given in the figure above.
- If we follow line marked 4,
 - P gets A.
 - Then Q starts executing and gets B.
 - Then P starts executing and releases A.
 - Then Q resumes execution and gets A, then releases B.
 - Now P resumes execution and gets B, then releases B.



Deadlock of processes over resources



- Two processes, P and Q, have the general form given in the figure above.
- If we follow line marked 5,
 - P gets A, then releases A, then gets B.
 - Then Q starts executing and requests B. Q is blocked.
 - Then P releases B. Now Q can get B and execute.



Deadlock of processes over resources

Process P	Process Q
...	...
Get A	Get B
...	...
Release A	Get A
...	...
Get B	Release B
...	...
Release B	Release A
...	...

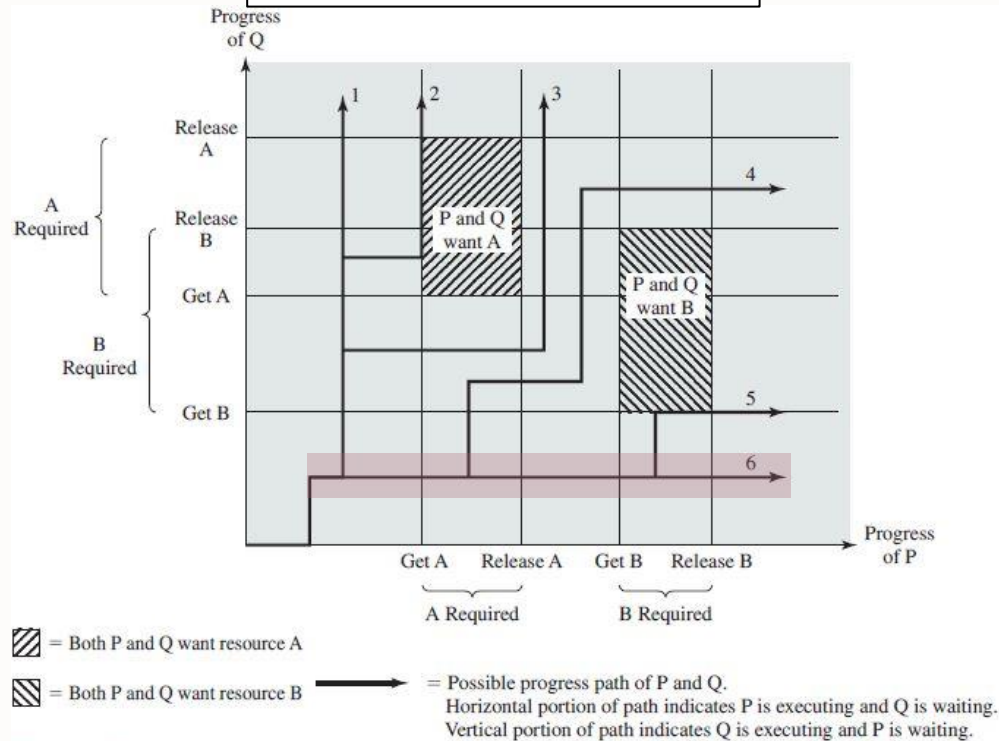


Figure 6.3 Example of No Deadlock [BACO03]

- Two processes, P and Q, have the general form given in the figure above.
- If we follow line marked 6,
 - P gets A, then releases A, then gets B, then releases B. Now Q can execute.



Deadlock of processes over resources

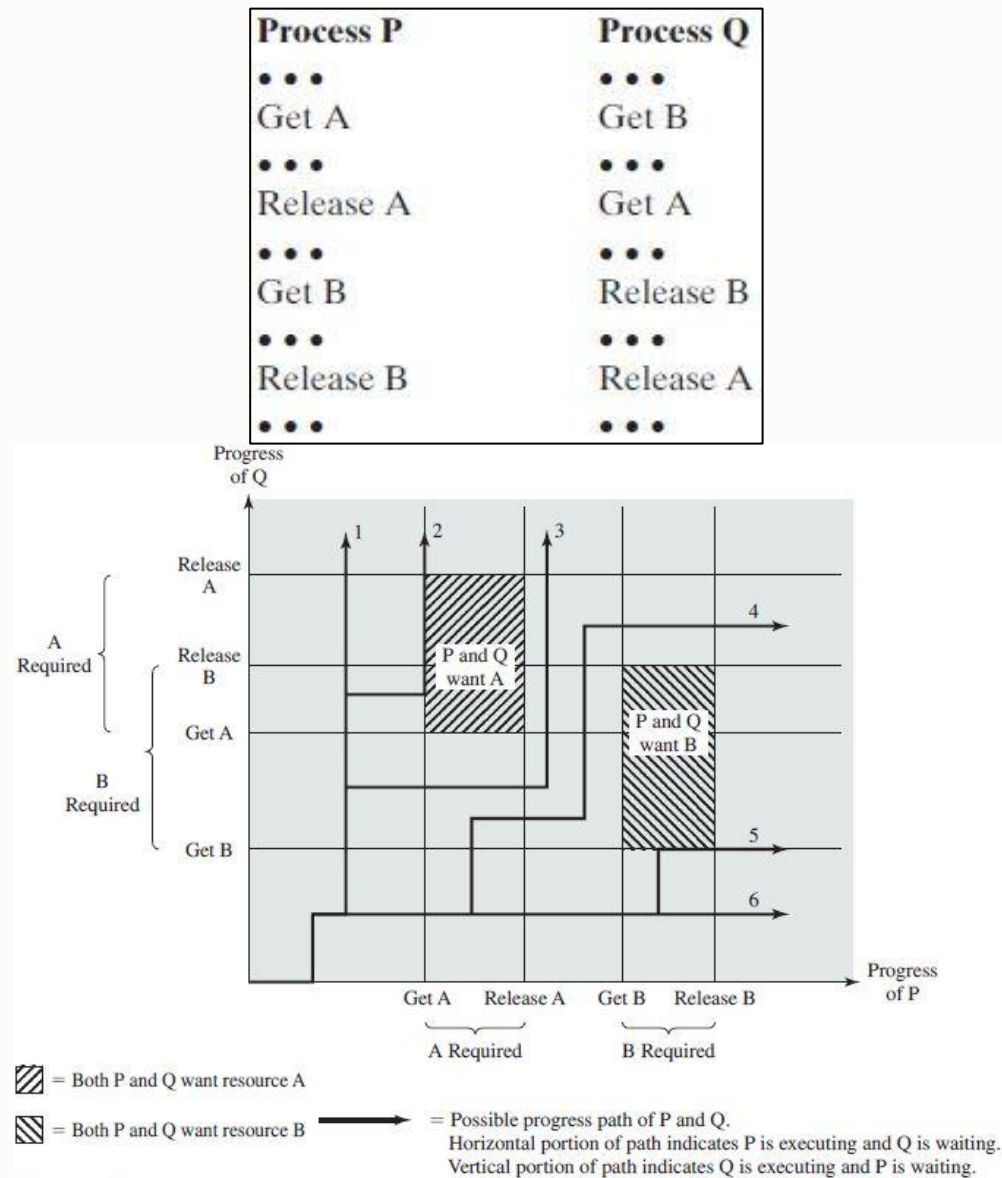


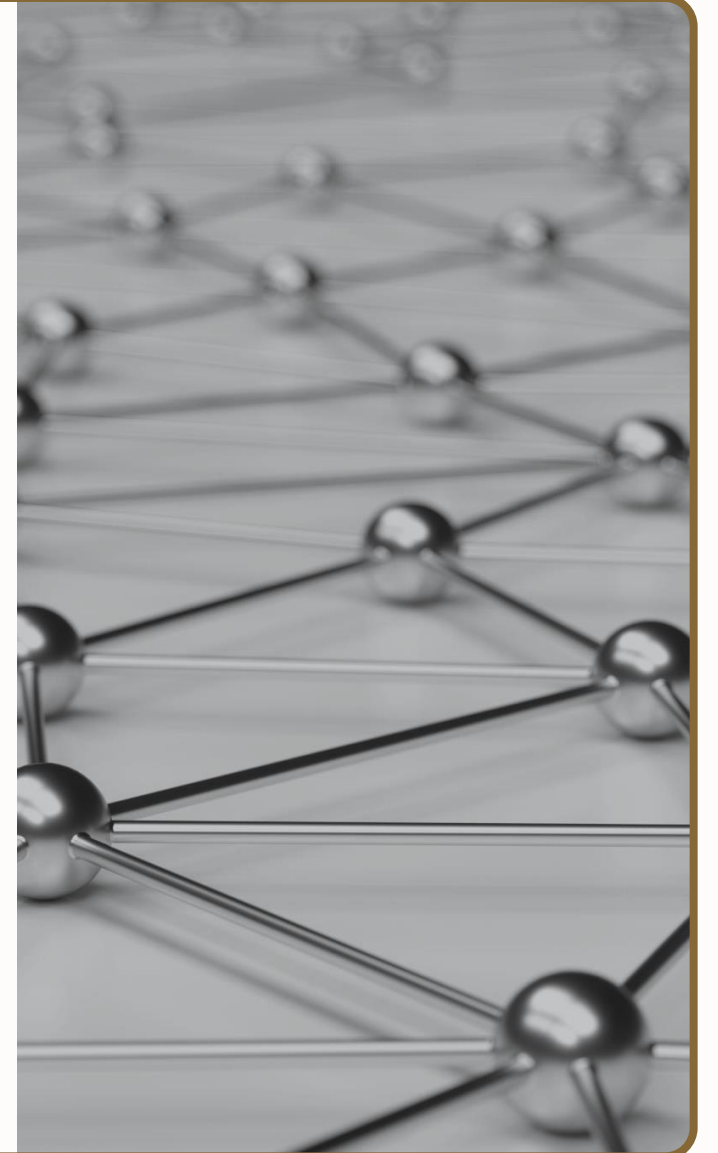
Figure 6.3 Example of No Deadlock [BACO03]

- Two processes, P and Q, have the general form given in the figure above.
- Regardless of the relative timing of the two processes, deadlock cannot occur.
- In cases where more than two processes may compete for the same resource, a higher-dimensional diagram would be required.



Types of resources

- Two general categories of resources can be distinguished:
 - Reusable resources, and
 - Consumable resources.



Reusable resources

- A reusable resource is one that can be safely used by only one process at a time and is not depleted by that use.
- Processes obtain resource units that they later release for reuse by other processes.
- Examples of reusable resources include processors; I/O channels; main and secondary memory; devices; and data structures such as files, databases, and semaphores.



Deadlock involving reusable resources

Step	Process P Action	Step	Process Q Action
p ₀	Request (D)	q ₀	Request (T)
p ₁	Lock (D)	q ₁	Lock (T)
p ₂	Request (T)	q ₂	Request (D)
p ₃	Lock (T)	q ₃	Lock (D)
p ₄	Perform function	q ₄	Perform function
p ₅	Unlock (D)	q ₅	Unlock (T)
p ₆	Unlock (T)	q ₆	Unlock (D)

Figure 6.4 Example of Two Processes Competing for Reusable Resources

- Two processes compete for exclusive access to a disk file D and a tape drive T.
- Deadlock occurs if each process holds one resource and requests the other.
- Deadlock occurs if the multiprogramming system interleaves the execution of the two processes as follows:

p₀ p₁ q₀ q₁ p₂ q₂



Deadlock involving main memory

P1	P2
...	...
Request 80 Kbytes;	Request 70 Kbytes;
...	...
Request 60 Kbytes;	Request 80 Kbytes;

- Suppose the space available for allocation is 200 Kbytes.
- The sequence of requests that occurs is depicted in the picture.
- Deadlock occurs if both processes progress to their second request.
- The best way to deal with this problem is by using virtual memory.

for line 2 since $70+80=150$ Kbytes so now $200-150=50$ B left cannot cater any of the request both will expect other to leave



A consumable resource is one that can be created (produced) and destroyed (consumed).

Typically, there is no limit on the number of consumable resources of a type.

An unblocked producing process may create any number of such resources.

When a resource is acquired by a consuming process, the resource ceases to exist.

Examples of consumable resources are interrupts, signals, messages, and information in I/O buffers.

Consumable resources



Deadlock involving consumable resources

P1	P2
...	...
Receive (P2);	Receive (P1);
...	...
Send (P2, M1);	Send (P1, M2);

each process attempts to receive a message from other, since receive is blocking, deadlock

- Consider the pair of processes shown in the figure, in which each process attempts to receive a message from the other process and then send a message to the other process.
- Deadlock occurs if the Receive is blocking (i.e., the receiving process is blocked until the message is received).



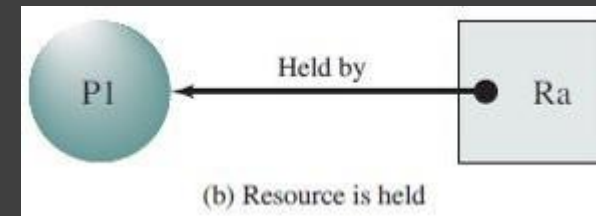
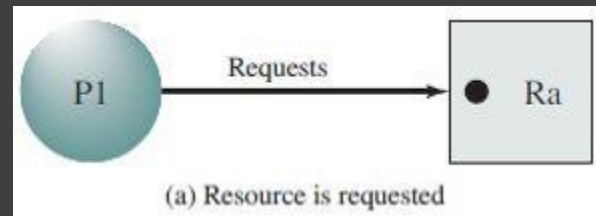
Resource allocation graphs

- A useful tool in characterizing the allocation of resources to processes is the resource allocation graph.
- The resource allocation graph is a directed graph that depicts a state of the system of resources and processes, with each process and each resource represented by a node.
- Within a resource node, a dot is shown for each instance of that resource.
- Examples of resource types that may have multiple instances are I/O devices that are allocated by a resource management module in the OS.

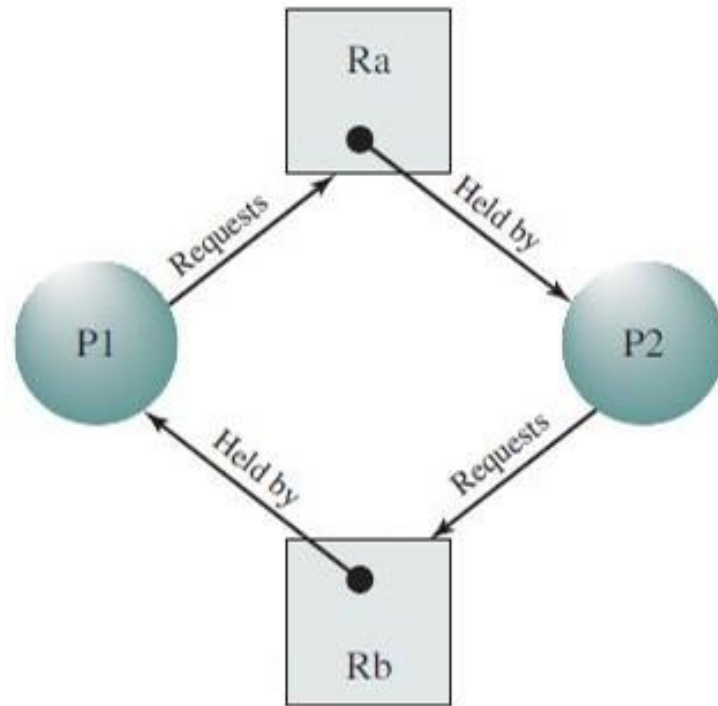


Resource allocation graphs

- A graph edge directed from a process to a resource indicates a resource that has been requested by the process but not yet granted.
- A graph edge directed from a reusable resource node dot to a process indicates a request that has been granted: that is, the process has been assigned one unit of that resource.
- A graph edge directed from a consumable resource node dot to a process indicates that the process is the producer of that resource.



A resource allocation graph with deadlock

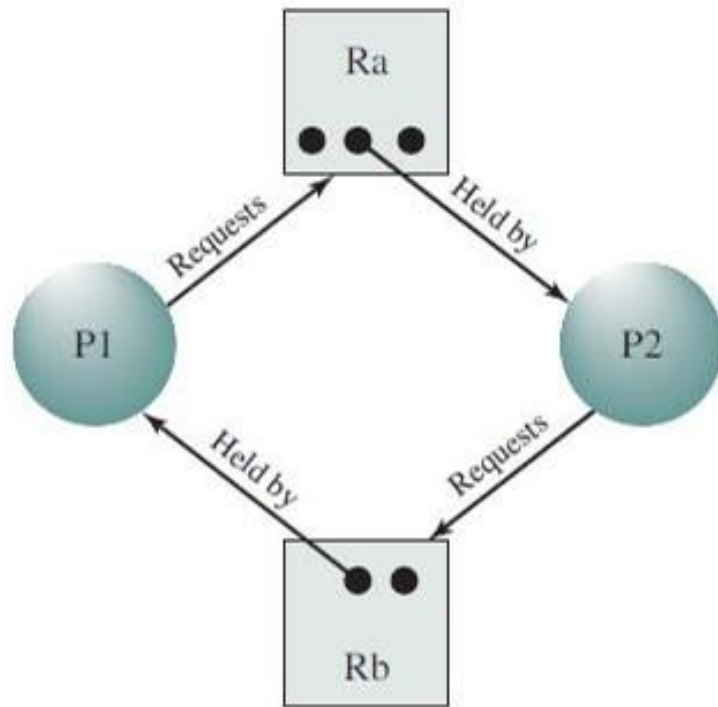


(c) Circular wait

- There is only one unit each of resources R_a and R_b .
- Process P_1 holds R_b and requests R_a , while P_2 holds R_a but requests R_b .



A resource allocation graph without deadlock

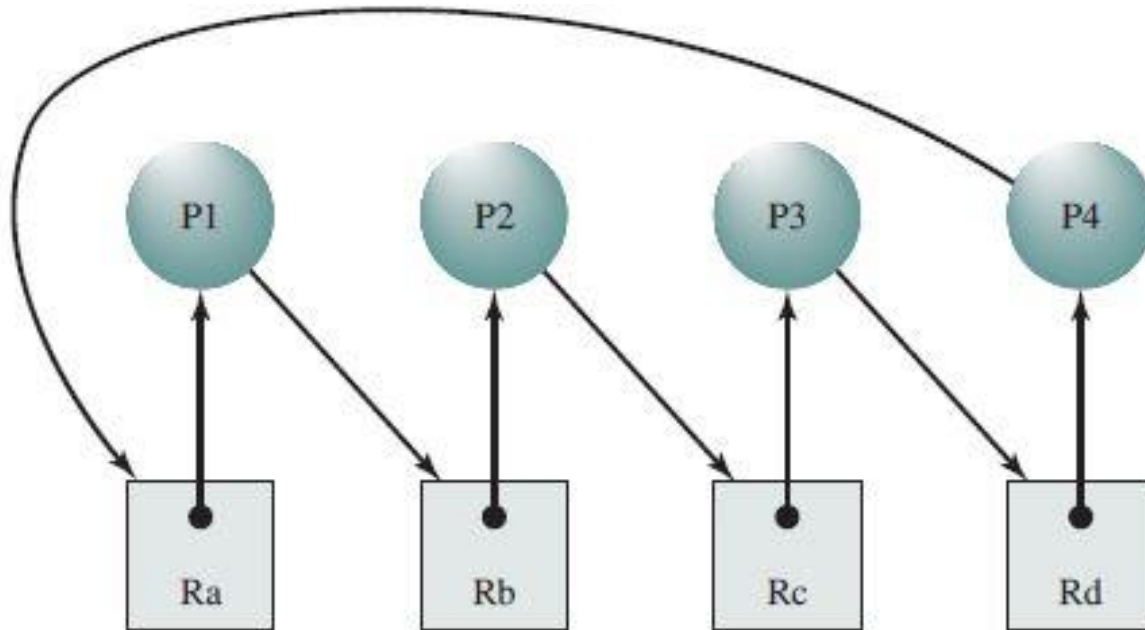


(d) No deadlock

- There are three units of R_a and two units of R_b .
- Process P_1 holds R_b and requests R_a , while P_2 holds R_a and requests R_b .
- There is no deadlock as multiple instances of resources are available.



A resource allocation graph with deadlock



- Note that in this case, we do not have a simple situation in which two processes each have one resource the other needs.
- Rather, in this case, there is a circular chain of processes and resources that results in deadlock.

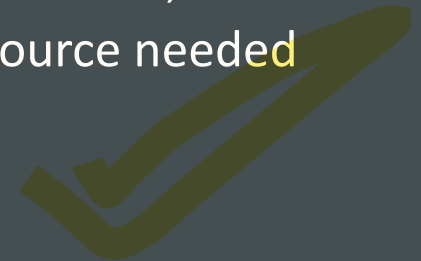




The conditions for deadlock



- Three conditions of policy must be present for a deadlock to be possible:
 - **Mutual exclusion**: Only one process may use a resource at a time. No process may access a resource unit that has been allocated to another process.
 - **Hold and wait**: A process may hold allocated resources while awaiting assignment of other resources.
 - **No preemption**: No resource can be forcibly removed from a process holding it.
- The first three conditions are necessary but not sufficient for a deadlock to exist. For deadlock to take place, we require:
 - **Circular wait**: A closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain.



Necessary and sufficient conditions for deadlock

Possibility of Deadlock	Existence of Deadlock
1. Mutual exclusion 2. No preemption 3. Hold and wait	1. Mutual exclusion 2. No preemption 3. Hold and wait 4. Circular wait

- The fourth condition is a potential consequence of the first three. That is, given that the first three conditions exist, a sequence of events may occur that lead to an unresolvable circular wait.
- The unresolvable circular wait is in fact the definition of deadlock.
- The circular wait is unresolvable because the first three conditions hold.
- The four conditions constitute necessary and sufficient conditions for deadlock.



Pause the video!

Write your answer in a word file.
Save it as
<my_roll_no>_lecture_24. If
your roll no is 500, the file will
be named 500_lecture_24. Do
not add <> signs to the
filename.

Task# 1: What is a necessary condition?

me. hold and wait, non pre emptio

Task# 2: What is a sufficient condition?

circular wait



Approaches for dealing with deadlock

- Three general approaches exist for dealing with deadlock.
 1. One can **prevent** deadlock by adopting a policy that eliminates one of the four conditions.
 2. One can **avoid** deadlock by making the appropriate **dynamic choices** based on the current state of resource allocation.
 3. One can attempt to **detect** the presence of deadlock and take action to **recover**.



Deadlock Prevention

Nullifying one of the four conditions of
deadlock



Methods of deadlock prevention

- The strategy of deadlock prevention is to design a system in such a way that the possibility of deadlock is excluded.
- We can view deadlock prevention methods as falling into two classes.
- An indirect method of deadlock prevention is to prevent the occurrence of one of the three necessary conditions listed previously (ME, hold and wait, no preemption).
- A direct method of deadlock prevention is to prevent the occurrence of a circular wait.



Nullifying mutual exclusion

- In general, the first of the four listed conditions cannot be nullified.
- If access to a resource requires mutual exclusion, then mutual exclusion must be supported by the OS.
- Some resources, such as files, may allow multiple accesses for reads but only exclusive access for writes.
- Even in this case, deadlock can occur if more than one process requires write permission.



Nullifying hold and wait

A process may request and get resources early, but it does not use them immediately in every part of its program.

So:

The resource stays allocated to that process.
Other processes cannot use it meanwhile.
The resource remains idle (unused) for some time.

(this is the second way)

- The hold-and-wait condition can be prevented by requiring that a process request all its required resources at one time and blocking the process until all requests can be granted simultaneously.
- This approach is inefficient in two ways:
 1. A process may be held up for a long time waiting for all its resource requests to be filled, when in fact it could have started execution with only some of the resources.
 2. Resources allocated to a process may remain unused for a considerable period, during which time they are denied to other processes. This is because the process uses a resource in only some of its total lines of code.

hold and wait : A process holds one resource while waiting for another.



Nullifying hold and wait

-
- Another problem is that a process may not know in advance all the resources that it will require.
 - There is also the practical problem created by the use of modular programming or a multithreaded structure for an application. An application would need to be aware of all resources that will be requested at all levels or in all modules to make the simultaneous request. But in modular or multithreaded programming, this information may be difficult to get by.

Nullifying no preemption

- These protocols are practical only when applied to resources whose state can be easily saved and restored later, as is the case with a processor.
- No preemption condition can be prevented in several ways:
 1. If a process holding certain resources is denied a further request, that process must release its original resources and, if necessary, request them again together with the additional resource.
 2. If a process requests a resource that is currently held by another process, the OS may preempt the second process and require it to release its resources. This scheme would prevent deadlock only if the two processes do not possess the same priority.

ya toh isi ko processor sai hata do ya jisky pass hai us sai lekar uskoutaar kar resource siko dedo



Nullifying circular wait

- The circular-wait condition can be prevented by **defining a linear ordering of resource types**. We associate an index with each resource type. If a process has been allocated resource R_i , then it may subsequently request R_x such that $x > i$.
- Resource R_i precedes R_j in the ordering as $i < j$.
- Suppose that a process A has acquired R_i and a process B has acquired R_j .
- A requests R_j and is blocked.
- B wants to request R_i but cannot as $i < j$. So B will have to relinquish R_j .
- A can now be served.
- As with hold-and-wait prevention, circular-wait prevention may be inefficient, slowing down processes and denying resource access unnecessarily.



Lessons from the lecture

We understood what a deadlock is.

We realized that there are two types of resources in a computer system.

We explored the conditions for deadlock.

We listed the approaches to deal with deadlock.

We explored deadlock prevention.



Things to do

